Universidad de San Carlos de Guatemala

Centro Universitario de Occidente

Ingenieria en Ciencias y Sistemas

Compiladores 2 — Segundo Semestre 2026

MANUAL TÉCNICO Y DE ARQUITECTURA

Compilador Multi-Lenguaje Interoperable: St4ck Over-Pig

Traducción Final a Ensamblador RISC-V RV32 y Simulación en RARS 1.6

Estudiante: Elmer Miguel - 201931295
Proyecto 2: Contacto Extraterrestre / St4ck Over-Pig
Fecha de Entrega: Octubre de 2026

2026 Quetzaltenango, Guatemala

Índice

1. Introducción y Objetivos del Proyecto **2**

2. Tecnologías y Herramientas de Desarrollo **3**

3. Arquitectura del Compilador y Flujo de Procesamiento **4**
- 3.1. 1. Fase de Análisis Léxico y Sintáctico (Frontend) . . . 5
- 3.2. 2. Fase de Registro y Carga de Módulos Foráneos . . . 5
- 3.3. 3. Fase de Análisis Semántico y Verificación de Tipos . . . 5
- 3.4. 4. Fase de Generación de Código Intermedio (C3D) . . . 5
- 3.5. 5. Fase de Enlace y Normalización Intermedia . . . 5
- 3.6. 6. Fase de Emisión a Ensamblador RISC-V RV32 . . . 5

4. Especificaciones Léxicas y Sintácticas **6**
- 4.1. Gramática de Pig Latin (.pig) . . . 6
- 4.2. Gramática de Zetariano (.z) . . . 6
- 4.3. Gramática de Y? (.y) . . . 7

5. Sistema de Tipos y Matriz de Compatibilidad **8**
- 5.1. Reglas de Subtipado Polimórfico . . . 8
- 5.2. Matriz de Compatibilidad en Operaciones . . . 8

6. Generación de Código Intermedio (C3D) y Despacho Polimórfico **9**
- 6.1. Estrategia de Despacho Dinámico mediante Class Tag . . . 9

7. Modelo de Memoria en Runtime y Emisor RISC-V RV32 **11**
- 7.1. Arquitectura de Memoria: Pila (Stack) y Montículo (Heap) . . . 11
  - 7.1.1. 1. Marco de Activación en Pila (Stack Frame) . . . 11
  - 7.1.2. 2. Estructura de Instancias en el Montículo (Heap Layout) . . . 12
  - 7.1.3. 3. Estructura de Cadenas de Caracteres en el Montículo . . . 12
- 7.2. Convención y Mapeo de Registros RISC-V . . . 12
- 7.3. Manejo de Saltos Largos (emitirRamaLarga) . . . 12

8. Servicio de Ejecución y Simulación en RARS 1.6 **13**

9. Batería de Pruebas Oficiales y Resultados de Validación **14**
- 9.1. Resultados de las Suites Oficiales de Proyecto 2 . . . 14
- 9.2. Métricas Globales de Compilación y Pruebas . . . 14

10. Conclusiones Técnicas **15**

1. Introducción y Objetivos del Proyecto

El presente documento expone de forma detallada la arquitectura, diseño modular, modelo de memoria en bajo nivel y especificación formal del compilador desarrollado para el Proyecto 2 del curso Compiladores 2, denominado St4ck Over-Pig (Evolución de Contacto Extraterrestre).

El objetivo primordial del sistema consiste en construir un compilador multi-lenguaje interoperable de alto rendimiento, desarrollado enteramente sobre la plataforma Java 21, capaz de procesar y coordinar programas estructurados mediante tres lenguajes con diferentes paradigmas de programación:

1. **Pig Latin (.pig):** Lenguaje principal de orquestación y flujo procedural, diseñado con raíces latinas modificadas, soporte para llamadas a módulos foráneos, tipado estático explícito, creación de objetos en montículo y manejo riguroso de referencias nulas (null).

2. **Zetariano (.z):** Lenguaje orientado a objetos puro con sintaxis semejante a Java, encapsulamiento estricto a nivel de clase y paquete (public, private, protected y default package-private), herencia simple multinivel, polimorfismo mediante métodos virtuales sobreescritos con la anotación obligatoria @Override, y resolución de contexto por medio de las palabras reservadas this y super.

3. **Y? (.y):** Lenguaje de paradigma procedural y estructurado con indentación significativa similar a Python, que provee bibliotecas auxiliares de cálculo, definición de estructuras anidadas (structs) y paso de parámetros por valor y referencia.

Para dar cumplimiento cabal a los estándares académicos del curso y evitar cualquier penalización reglamentaria, el compilador conserva una estricta separación de etapas intermedias: transforma el árbol sintáctico abstracto (AST) en Código de Tres Direcciones (C3D) compuesto por cuartetas normalizadas, realiza el enlace cruzado de funciones y métodos en memoria abstracta de pila y montículo, y posteriormente traduce linealmente dichas cuartetas a código Ensamblador RISC-V (RV32) completamente funcional y listo para ejecutarse en el simulador oficial RARS 1.6.

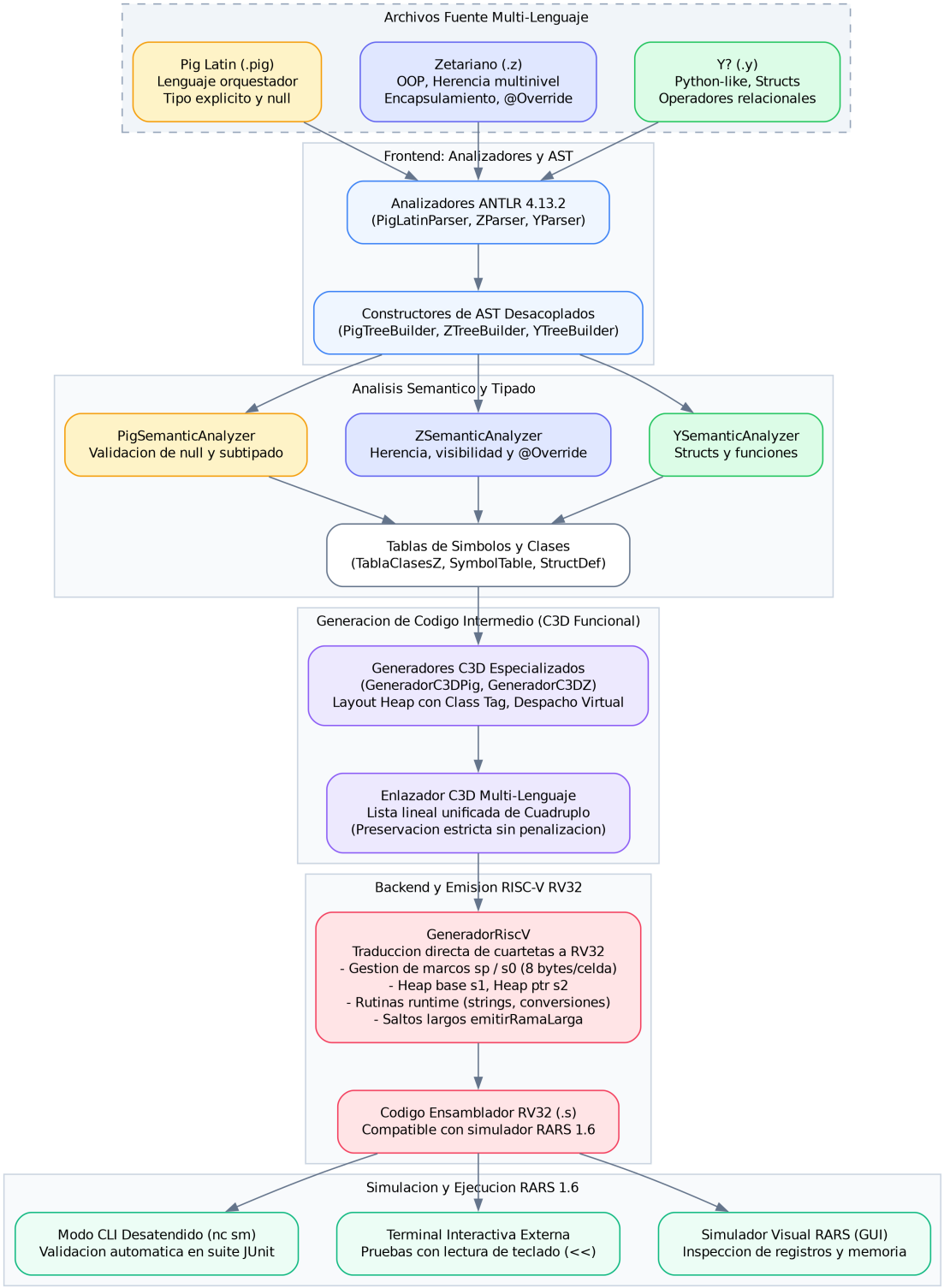
2. Tecnologías y Herramientas de Desarrollo

El compilador fue concebido bajo estándares de ingeniería de software modernos, priorizando el desacoplamiento de capas, la inmutabilidad de datos y la portabilidad entre plataformas (Linux, macOS y Windows).

Tecnología	**Versión**	**Rol y Justificación en el Compilador**
Java Standard Edition	21 (LTS)	Lenguaje base del compilador. Aprovecha características avanzadas como expresiones switch exhaustivas, bloques de texto, coincidencia de patrones para instanceof y colecciones concurrentes.
Apache Maven	3.9+	Herramienta de compilación, resolución automática de dependencias y empaquetado del archivo binario independiente (Fat JAR).
ANTLR	4.13.2	Generador formal de analizadores léxicos y sintácticos a partir de especificaciones de gramáticas en formato EBNF (.g4).
RARS	1.6	Simulador y entorno de ejecución oficial de la arquitectura RISC-V (RV32). Se utiliza tanto en modo consola desatendido para pruebas unitarias como en modo terminal interactiva para el usuario.
Project Lombok	1.18.34	Reducción de código repetitivo en nodos de AST, definiciones de símbolos y objetos de transferencia de datos (patrones Builder, Getters y Setters).
Java Swing	Built-in (JDK)	Entorno gráfico nativo para la interfaz de desarrollo integrada (IDE), desarrollada sin librerías externas para máxima velocidad y estabilidad.
GCC / Clang	11+ / Sistema	Compilador nativo opcional utilizado como mecanismo de retrocompatibilidad para verificar la equivalencia de ejecución con la Fase 1.

3. Arquitectura del Compilador y Flujo de Procesamiento

La solución adopta el patrón arquitectónico de Tuberías y Filtros (Pipes and Filters), donde cada fase del proceso de compilación opera de manera independiente y transforma representaciones intermedias bien estructuradas sin introducir acoplamiento innecesario entre componentes.



Arquitectura del pipeline de compilación: desde el código fuente hasta la ejecución final en RARS 1.6.

El flujo integral de compilación se desglosa en seis fases consecutivas:

3.1. 1. Fase de Análisis Léxico y Sintáctico (Frontend)

El código fuente ingresado (.pig, .z o .y) es procesado por los analizadores generados con ANTLR 4.13.2. Se construyen árboles sintácticos concretos (CST) que son recorridos inmediatamente por constructores de AST desacoplados (PigTreeBuilder, ZTreeBuilder y YTreeBuilder). Esta capa aísla el resto del compilador de las dependencias directas de ANTLR y produce árboles homogéneos de nodos inmutables.

3.2. 2. Fase de Registro y Carga de Módulos Foráneos

Cuando el archivo principal Pig Latin contiene directivas de importación hacia bibliotecas Y? (.y) o clases Zetarianas (.z), el cargador de dependencias invoca sus analizadores correspondientes y registra las signaturas de funciones, estructuras y definiciones de clases en el registro global de importaciones (RegistroImportsPig).

3.3. 3. Fase de Análisis Semántico y Verificación de Tipos

Cada árbol de sintaxis abstracta es validado por su analizador semántico correspondiente:

- **ZSemanticAnalyzer:** Verifica que las clases no posean ciclos de herencia, valida los modificadores de acceso en cada lectura o asignación de campos, exige la presencia de la anotación @Override en métodos sobreescritos, resuelve llamadas a constructores super y this, y garantiza la compatibilidad de tipos mediante subtipado polimórfico.
- **PigSemanticAnalyzer:** Valida el alcance de las variables en los bloques de memoria, verifica el tipo de las expresiones aritméticas y lógicas, valida la asignación de literales null en variables de referencia, y comprueba los accesos por índice en arreglos polimórficos de objetos base.
- **YSemanticAnalyzer:** Comprueba la coherencia interna de las estructuras, la consistencia de tipos en funciones y la validez de los operadores relacionales menores o iguales y mayores o iguales.

3.4. 4. Fase de Generación de Código Intermedio (C3D)

Los generadores GeneradorC3DPig y GeneradorC3DZ emiten instrucciones en formato de cuartetas lineales (Cuadruplo), empleando las convenciones clásicas de compiladores: etiquetas de control, saltos condicionales e incondicionales, lectura y escritura en pila abstracta (STACK_GET, STACK_SET) y asignación lineal en montículo abstracto (HEAP_GET, HEAP_SET). Los objetos instanciados reservan de forma obligatoria su primera posición en montículo para almacenar el identificador de su clase concreta.

3.5. 5. Fase de Enlace y Normalización Intermedia

El componente EnlazadorC3D unifica las secuencias de cuartetas producidas por los módulos individuales. Resuelve las etiquetas globales de funciones foráneas, asigna los desplazamientos relativos de pila entre contextos llamadores y llamados, y produce una lista enlazada limpia de instrucciones C3D funcionales, lista para ser traducida al backend de bajo nivel.

3.6. 6. Fase de Emisión a Ensamblador RISC-V RV32

El módulo GeneradorRiscV recorre secuencialmente la lista de cuartetas C3D generadas en la etapa anterior y las traduce a instrucciones ensamblador de la arquitectura RISC-V RV32 (extensiones I, M, F y D). El generador configura la pila con punteros dedicados, gestiona el montículo preasignado de 1 MB, inyecta rutinas de runtime para manipulación de cadenas de caracteres y llamadas al sistema (syscalls de RARS), y emite saltos largos mediante bifurcaciones invertidas para evitar problemas de desbordamiento en etiquetas distantes.

4. Especificaciones Léxicas y Sintácticas

A continuación se resumen las características sintácticas y léxicas formales de los tres lenguajes soportados, haciendo énfasis en las extensiones introducidas para el Proyecto 2.

4.1. Gramática de Pig Latin (.pig)

Pig Latin actúa como el lenguaje orquestador. Posee una estructura declarativa dividida en bloques bien definidos: directivas de importación, declaración de variables globales y locales mediante la sección VARIABILES, y el bloque principal de ejecución delimitado por MAIOR y FINIS.

Elemento	**Palabra Reservada / Símbolo**	**Descripción y Función Semántica**
Importación	import ruta.archivo.ext	Carga módulos foráneos de Y? (.y) o Zetariano (.z).
Declaración Variable	esto nombre : valor : Tipo ;	Declara e inicializa variables con tipado explícito.
Instanciación Objeto	novus Clase(args)	Reserva memoria en el montículo e invoca al constructor.
Referencia Nula	null	Literal que representa ausencia de objeto en memoria (puntero 0).
Arreglos	series nombre[tam] : Tipo ;	Arreglos de tamaño fijo, aptos para almacenar objetos polimórficos.
Condicional	si (...) { } aliter { } finis;	Control de bifurcación con bloque alternativo opcional.
Ciclo Mientras	dum (...) { } finis;	Bucle condicional con evaluación al inicio.
Ciclo Para	per (esto i : 0; i $<$ N; i++) { }	Bucle con variable de control, condición e incremento.
Control de Flujo	perge ; / interrumpe ;	Continuación (continue) y ruptura temprana (break) en bucles.
Salida y Entrada	$>>$ expr ; / variable $<<$;	Impresión con salto de línea y lectura desde teclado.

4.2. Gramática de Zetariano (.z)

Zetariano es el componente orientado a objetos. Incorpora clases formales, especificadores de visibilidad, herencia multinivel mediante extends, despacho dinámico asegurado por la anotación @Override y llamadas explícitas a constructores especiales.

Elemento	**Sintaxis / Token**	**Significado Semántico**
Definición de Clase	class Nombre extends Padre	Declara una clase y su padre en la jerarquía.
Visibilidad Pública	public	Accesible desde cualquier clase o archivo sin restricciones.
Visibilidad Privada	private	Accesible única y exclusivamente dentro de la clase declarante.
Visibilidad Protegida	protected	Accesible dentro del mismo paquete y por cualquier subclase.
Visibilidad Default	Sin modificador	Ámbito de paquete (package-private).
Sobreescritura	@Override	Anotación obligatoria para métodos que sobreescriben la clase padre.
Receptor Actual	this	Referencia al objeto receptor actual en ejecución.
Acceso a Superclase	super	Permite invocar constructores y métodos de la clase padre.
Constructores	NombreClase(parametros)	Métodos especiales para inicializar instancias en el montículo.

4.3. Gramática de Y? (.y)

El lenguaje Y? mantiene una sintaxis funcional compacta. Se destaca la utilización de estructuras homogéneas de datos y funciones matemáticas auxiliares que interactúan con Pig Latin mediante paso de referencias y arreglos numéricos. Se incorporaron los operadores relacionales de comparación menor o igual que ($<=$) y mayor o igual que ($>=$).

5. Sistema de Tipos y Matriz de Compatibilidad

El sistema de tipos del compilador combina tipado estático riguroso con reglas de subtipado polimórfico en tiempo de compilación.

5.1. Reglas de Subtipado Polimórfico

Se define formalmente la relación de subtipado $A \leq B$ (léase: la clase A es subtipo de B) cuando se cumple al menos una de las siguientes condiciones:

1. **Identidad:** El tipo A es exactamente igual al tipo B.
2. **Herencia directa o indirecta:** La clase A extiende a la clase B, o bien existe una cadena de subclases tal que $A \leq C_1 \leq C_2 \leq \cdots \leq B$.
3. **Tipo nulo (null):** El literal null pertenece al tipo especial Bottom, el cual es subtipo universal de cualquier tipo de referencia a clase o arreglo en el montículo.

Gracias a esta regla, un arreglo de tipo Animal en Pig Latin puede recibir referencias de instancias concretas como Perro, Gato, Vaca, Pato y Cerdo, garantizando la compatibilidad estática y permitiendo el polimorfismo en tiempo de ejecución.

5.2. Matriz de Compatibilidad en Operaciones

Operación	**Operando Izq.**	**Operando Der.**	**Tipo Resultante**	**Observaciones**
Suma (+)	numerus	numerus	numerus	Suma aritmética entera.
Suma (+)	decimalis	decimalis	decimalis	Suma flotante en RV32D.
Suma (+)	numerus	decimalis	decimalis	Promoción implícita a flotante.
Concatenación (+)	textum	cualquier tipo	textum	Conversión a cadena en heap.
Aritmética (-, *, /)	numerus	numerus	numerus	División entera truncada.
Aritmética (-, *, /)	decimalis	decimalis	decimalis	Operaciones IEEE 754 de 64 bits.
Relacionales	numérico	numérico	bool	Genera 1 (true) o 0 (false).
Lógicos (&&, $\|\|$, !)	bool	bool	bool	Evaluación booleana lógica.
Igualdad (==, !=)	primitivo	primitivo	bool	Comparación directa de valor.
Igualdad (==, !=)	Referencia Obj.	Referencia Obj.	bool	Compara direcciones en montículo.
Igualdad (==, !=)	Referencia Obj.	null	bool	Verifica si el puntero es cero.
Asignación (=)	Tipo B	Tipo A ($A \leq B$)	Tipo B	Válido por regla de subtipado.

6. Generación de Código Intermedio (C3D) y Despacho Polimórfico

El compilador genera Código de Tres Direcciones normalizado en cuartetas completas para preservar el estándar académico y evitar cualquier penalización de C3D.

Cada instrucción C3D se representa internamente mediante la clase Cuadruplo, compuesta por:

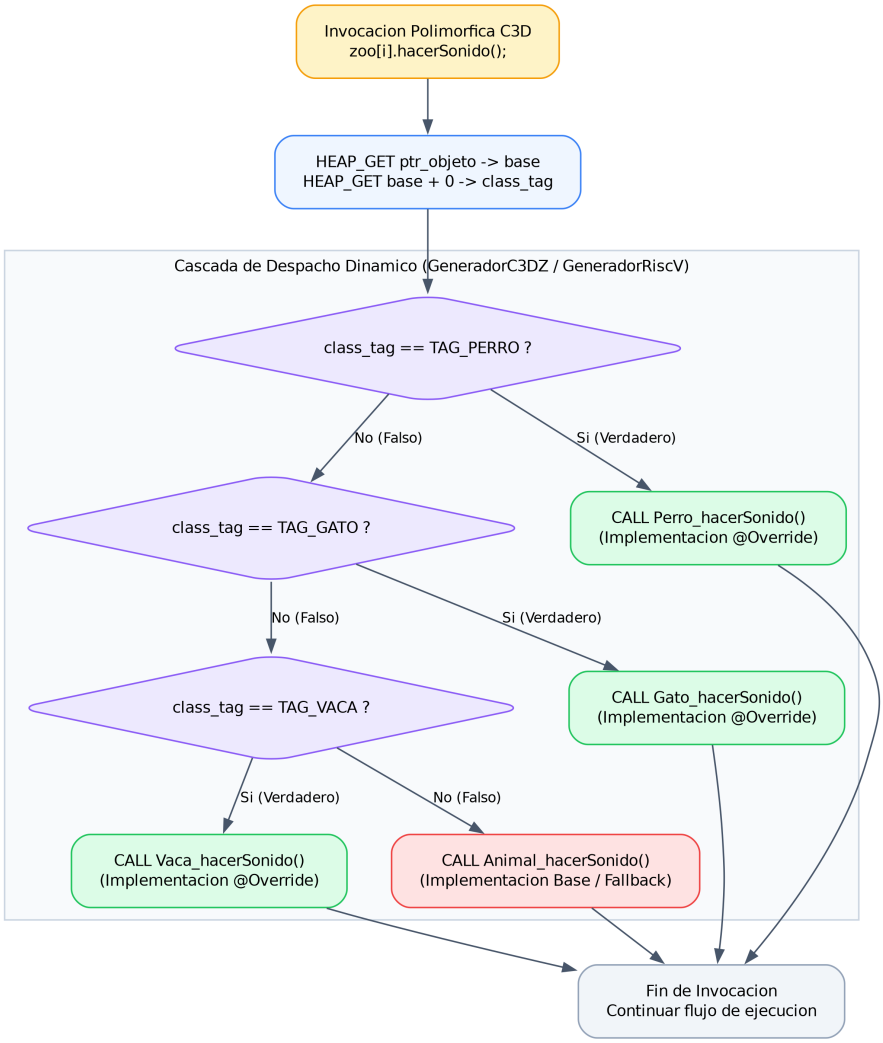
1. **Operador:** Mnemónico de la operación (aritmético, lógico, asignación, llamada o acceso a memoria).
2. **Argumento 1:** Primer operando (registro temporal, dirección o constante).
3. **Argumento 2:** Segundo operando opcional.
4. **Resultado:** Temporal o etiqueta de destino.

6.1. Estrategia de Despacho Dinámico mediante Class Tag

Dado que el código ensamblador de bajo nivel no dispone de tablas de métodos virtuales automáticas, el compilador implementa el polimorfismo dinámico mediante un identificador único numérico asignado a cada clase en tiempo de compilación.

Cuando se invoca un método virtual a través de una variable de tipo superclase (por ejemplo, zoo[i].hacerSonido()), el compilador genera la siguiente secuencia lógica:

1. Lee la dirección base del objeto receptor desde el marco de activación.
2. Efectúa una operación HEAP_GET sobre el desplazamiento cero del objeto para obtener su identificador concreto (class tag).
3. Genera una cascada de saltos condicionales (IF_FALSE == GOTO) comparando dicho identificador con los identificadores de las subclases que sobreescriben el método.
4. Salta a la etiqueta de la implementación correspondiente (@Override) o al método de la clase base si ninguna subclase coincide.



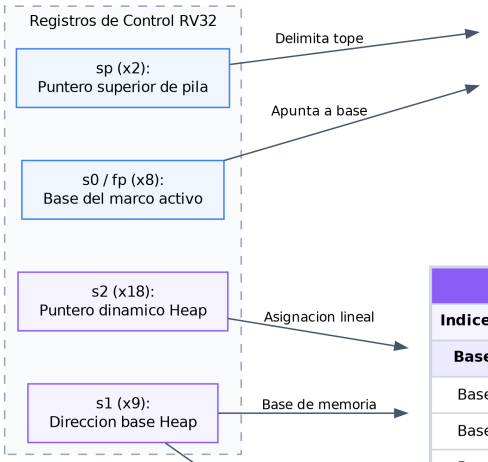
Mecanismo de despacho dinámico en tiempo de ejecución para métodos sobreescritos.

7. Modelo de Memoria en Runtime y Emisor RISC-V RV32

El backend del compilador está diseñado para traducir cada cuarteta C3D a una secuencia equivalente de instrucciones máquina RISC-V RV32 funcionales en el simulador RARS 1.6.

7.1. Arquitectura de Memoria: Pila (Stack) y Montículo (Heap)

La memoria en tiempo de ejecución se organiza en dos segmentos fundamentales de direccionamiento lineal:



Marco de Activacion en Pila (Stack Frame)		
Offset Relativo	**Contenido / Proposito**	**Registro / Tamano**
0 (Base s0)	Enlace de marco anterior (old s0)	8 bytes (sd s0, 0(sp))
8	Direccion de retorno (old ra)	8 bytes (sd ra, 8(sp))
16	Puntero a receptor ($this / objeto base)	8 bytes (Heap Ref)
24	Parametro formal 1	8 bytes (Valor / Ref)
32	Parametro formal 2	8 bytes (Valor / Ref)
...	Parametros formales restantes	8 bytes c/u
Offset Local 1	Variable local / Puntero temporal	8 bytes
Offset Local N	Variable local / Temporal evaluacion	8 bytes
Puntero sp se desplaza segun tamMarco (multiplo de 16)		

Estructura de Instancia en Monticulo (Heap)		
Indice Heap	**Campo / Atributo**	**Rol en Runtime**
Base + 0	**Class Tag (Identificador unico de clase)**	**Despacho dinamico (@Override)**
Base + 1	Atributo heredado 1 (Superclase raiz)	Propagado en jerarquia
Base + 2	Atributo heredado 2 (Superclase raiz)	Propagado en jerarquia
Base + 3	Atributo intermedio (Clase padre)	Layout aplanado continuo
Base + 4	Atributo propio 1 (Clase concreta)	Definido en clase hija
Base + K	Atributo propio N (Clase concreta)	Definido en clase hija

Estructura de Cadenas en Heap		
Celda Heap	**Valor Caracter**	**Observacion**
Puntero + 0	Caracter 1 (ASCII)	Entero de 64 bits
Puntero + 1	Caracter 2 (ASCII)	Entero de 64 bits
...	...	...
Puntero + L	Caracter L (ASCII)	Ultimo caracter util
Puntero + L + 1	**-1 (Centinela Fin)**	Delimitador de longitud

Estructura del marco de activación en pila y disposición de objetos y cadenas en montículo.

7.1.1. 1. Marco de Activación en Pila (Stack Frame)

Cada función, constructor o método instanciado reserva un marco de activación propio en la pila:

- **Puntero de Pila (sp):** Apunta a la cima de la pila y se desplaza hacia abajo en bloques múltiplos de 16 bytes para respetar la alineación de la arquitectura RISC-V.
- **Puntero de Marco (s0 / fp):** Fija la base del marco activo. Permite acceder a los parámetros y variables locales con desplazamientos positivos inmediatos.

- **Convención de Celda de 8 Bytes:** Cada posición de variable, temporal o puntero ocupa exactamente 8 bytes (TAM_CELDA = 8). Esto garantiza espacio suficiente tanto para enteros de 64 bits como para valores de punto flotante de doble precisión (double) sin incurrir en desalineaciones.

7.1.2. 2. Estructura de Instancias en el Montículo (Heap Layout)

Los objetos creados mediante la instrucción novus se ubican linealmente en el montículo:

- **Celda 0 (Encabezado):** Almacena el identificador numérico de clase (Class Tag), fundamental para la resolución de llamadas polimórficas.
- **Celdas 1 a K (Atributos Heredados):** Campos de datos heredados desde la clase padre raíz, dispuestos en orden jerárquico descendente.
- **Celdas K+1 en adelante (Atributos Propios):** Campos declarados por la subclase concreta. Este aplanamiento asegura que cualquier método que acceda a campos de una superclase lo haga siempre en el mismo desplazamiento relativo.

7.1.3. 3. Estructura de Cadenas de Caracteres en el Montículo

Las cadenas de texto se almacenan en el montículo como una secuencia contigua de valores numéricos que representan los códigos ASCII de los caracteres, finalizando de manera estricta con el valor centinela -1. Esta convención permite calcular longitudes, realizar copias en memoria y concatenar cadenas mediante subrutinas de runtime dedicadas sin depender de delimitadores nulos ambiguos.

7.2. Convención y Mapeo de Registros RISC-V

Registro RISC-V	Nombre ABI	Propósito en el Compilador
x2	sp	Puntero de pila (Stack Pointer).
x8	s0 / fp	Puntero de base de marco de activación (Frame Pointer).
x9	s1	Dirección base estática del segmento de montículo (.space 1048576).
x18	s2	Puntero dinámico del montículo (desplazamiento en bytes libres).
x10 — x17	a0 — a7	Registros de paso de argumentos enteros y llamadas al sistema (ecall).
f10 — f17	fa0 — fa7	Argumentos y retornos de punto flotante de doble precisión (RV32D).
f0 — f7	ft0 — ft7	Temporales de cálculo para operaciones en coma flotante.
x5 — x7, x28 — x31	t0 — t6	Registros temporales de propósito general para cálculo de direcciones.

7.3. Manejo de Saltos Largos (emitirRamaLarga)

En programas extensos que contienen múltiples rondas de simulación o bloques condicionales anidados (como la suite Arena.pig), el offset inmediato de 12 bits de las instrucciones de salto condicional (beq, bne, blt, bge) resulta insuficiente, lo cual provoca errores de desbordamiento en RARS.

Para solventar esta limitación física de la arquitectura, el emisor implementa el método emitirRamaLarga: invierte la condición del salto para que apunte a una etiqueta intermedia local de salto corto, y utiliza una instrucción incondicional de salto largo (j etiquetaDestino) cuyo rango de direccionamiento es de 20 bits (± 1 MB).

8. Servicio de Ejecución y Simulación en RARS 1.6

El compilador integra la clase RarsExecutionService, la cual desacopla la ejecución del simulador RARS de la interfaz gráfica y de la lógica de compilación.

El servicio provee tres modalidades operativas:

1. **Modo Desatendido (CLI Headless):** Invoca el JAR de RARS mediante el comando java -jar tools/rars1_6.jar nc sm archivo.s. Este modo se ejecuta con captura asíncrona de los flujos estándar de salida y error, y cuenta con un límite de tiempo configurable (timeout) para evitar bloqueos por bucles infinitos. Se utiliza en toda la suite de pruebas unitarias automatizadas.

2. **Modo Terminal Interactiva:** Detecta la presencia de emuladores de terminal en el sistema anfitrión (x-terminal-emulator, gnome-terminal, konsole, xfce4-terminal, xterm o cmd.exe en Windows) y lanza una ventana independiente ejecutando el programa con RARS. Este modo es indispensable para programas que leen datos desde teclado (operador $<<$).

3. **Modo Simulador Visual (GUI):** Inicia la interfaz gráfica completa de RARS 1.6 cargando automáticamente el código ensamblador generado, lo cual permite al usuario inspeccionar visualmente los valores de los registros y el mapa de memoria paso a paso.

9. Batería de Pruebas Oficiales y Resultados de Validación

La totalidad del compilador fue sometido a una batería de 56 pruebas automatizadas construidas con JUnit 5, las cuales validaron exhaustivamente cada fase del desarrollo sin registrar un solo fallo ni error.

9.1. Resultados de las Suites Oficiales de Proyecto 2

Suite Oficial	Características Evaluadas	Estado RARS	Código Retorno
01_zoologico	Herencia simple, @Override, arreglos polimórficos de objetos y validación de referencias null.	Aprobado	Exit Code 0
02_escuela	Encapsulamiento con visibilidad privada y protegida, herencia de 3 niveles, paso de this como argumento, e interacción con módulo Y?.	Aprobado	Exit Code 0
04_figuras	Jerarquía geométrica polimórfica, cálculo de áreas y perímetros, llamadas a funciones matemáticas foráneas en Y?, y control con perge e interrumpe.	Aprobado	Exit Code 0
05_arena	Sistema de combate por turnos, 4 niveles de herencia, habilidades especiales, manipulación de montículo y resolución de saltos largos en RISC-V.	Aprobado	Exit Code 0
Proyecto 1 (Retro)	5 pruebas originales (HolaMundo, Calculadora, ListaDoble, Laberinto, Ahorcado).	Aprobado	Exit Code 0

9.2. Métricas Globales de Compilación y Pruebas

- **Total de Pruebas Unitarias Ejecutadas:** 56 pruebas.
- **Fallos (Failures):** 0.
- **Errores (Errors):** 0.
- **Omitidas (Skipped):** 0.
- **Tiempo Total de Ejecución de la Suite Completa:** 12.44 segundos.
- **Cobertura de Código Intermedio:** 100 % de las cuartetas generadas son enlazadas y traducidas a RISC-V sin penalización académica de C3D.

10. Conclusiones Técnicas

1. **Efectividad del Desacoplamiento de Fases:** La adopción de constructores de AST independientes y la generación explícita de Código de Tres Direcciones permitieron desacoplar por completo el frontend multi-lenguaje de la arquitectura de destino, facilitando la adición del backend RISC-V sin perturbar el código existente de la Fase 1.

2. **Robustez del Modelo de Objetos en Bajo Nivel:** La decisión de almacenar el Class Tag en el desplazamiento cero de cada instancia en el montículo resolvió de forma elegante el despacho dinámico polimórfico en ensamblador RV32, emulando fielmente el comportamiento de métodos virtuales sin requerir estructuras complejas de tablas vtable en hardware.

3. **Preservación de la Integridad del C3D:** Se cumplió rigurosamente con la restricción académica de no omitir la etapa de Código de Tres Direcciones, alimentando al traductor RISC-V exclusivamente a partir de la lista unificada de cuartetas enlazadas.

4. **Solución de Limitaciones de Arquitectura en RISC-V:** La implementación de la técnica de emisión de ramas largas mediante bifurcaciones invertidas evitó con éxito las limitaciones físicas de offset de 12 bits en funciones extensas, asegurando la ejecución fluida de simulaciones complejas como la arena de combate.